Case Study 2: Progressive Electricity Billing

Activity sheet case study #2 | Concepts: Variables, operators, conditional statements, loops, lists (parallel lists), formatted strings

1. Problem Statement

A household electricity bill is computed from consumption slabs, a fixed charge, a surcharge for heavy usage and a rebate for low usage. Write a program that computes the bill for several months, compares each month with the previous month and flags unusual increases.

2. Objectives

- Implement slab-wise (progressive) tariff calculation using loops over lists.
- Apply a surcharge and a rebate using conditional statements.
- Compare consecutive months and compute percentage change.
- Validate readings and summarise total, average and highest bill.

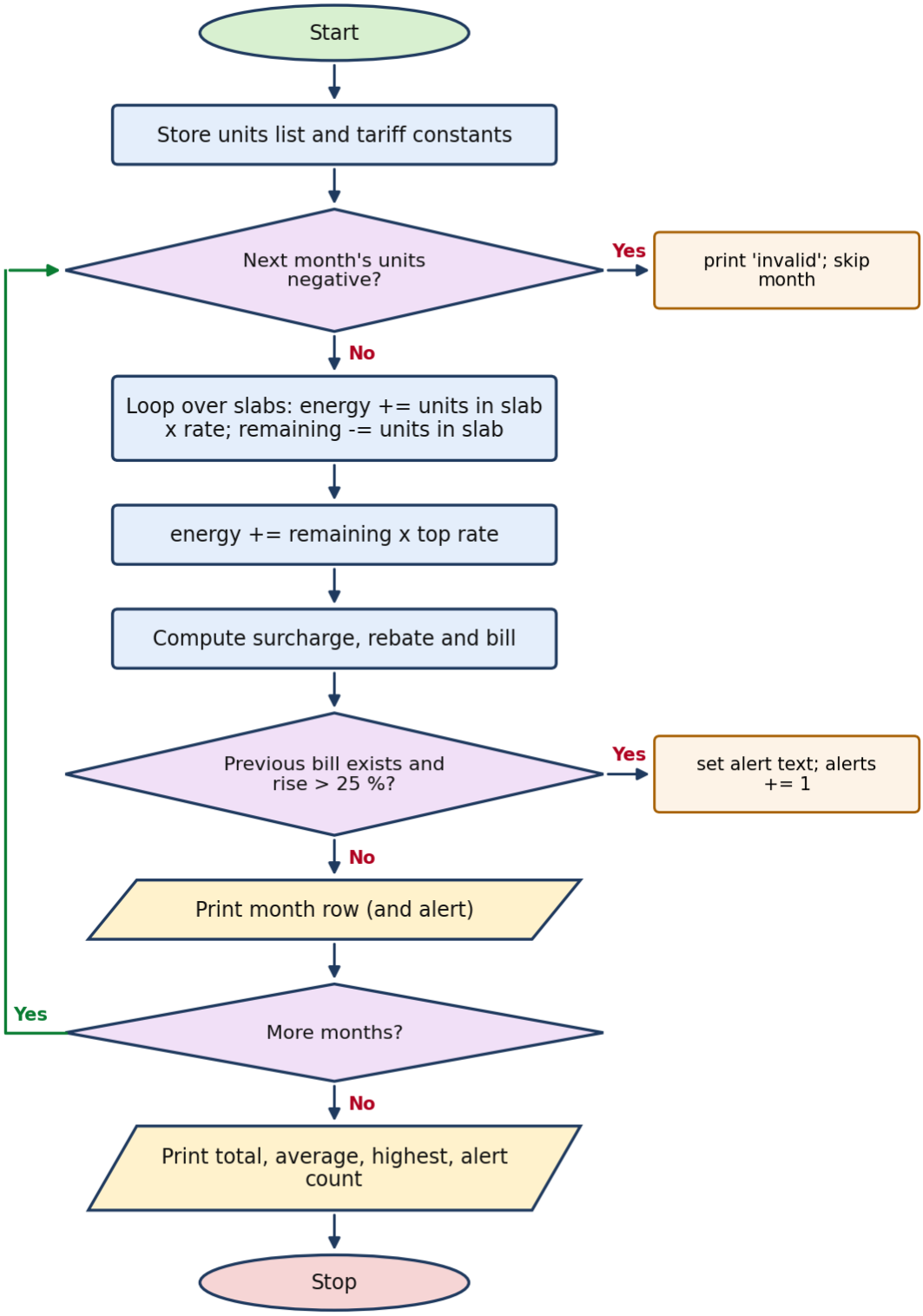
3. Assumptions

- Slabs: first 100 units at Rs 3.50, next 100 at Rs 5.00, next 100 at Rs 7.00, above 300 units at Rs 9.00.
- Fixed charge is Rs 120 per month.
- Surcharge of 8 % of the energy charge applies when units exceed 300.
- Rebate of 5 % of the energy charge applies when units are 100 or fewer.
- A bill is 'unusual' when it rises by more than 25 % over the previous valid month.
- Negative readings are invalid and skipped.

4. Algorithm

1. Store the monthly units list and tariff constants.
2. For each month, read the units; if negative, report an invalid reading and skip the month.
3. Initialise remaining = units and energy = 0; for every slab take min(remaining, slab size), multiply by the slab rate, add to energy and reduce remaining.
4. Charge the leftover units at the top rate.
5. Compute surcharge (units > 300) and rebate (units <= 100).
6. bill = energy + fixed + surcharge - rebate.
7. If a previous bill exists, compute percentage change and flag it when above 25 %.
8. Update running total, highest bill and previous bill; repeat for all months.
9. Print total, average, highest bill and the number of flagged months.

5. Flowchart



6. Pseudocode

```
START
  SET slab sizes = [100,100,100], rates = [3.5,5.0,7.0], TOP_RATE = 9.0
  SET FIXED = 120, previous_bill <- -1
  FOR each month i with units
      IF units < 0 THEN PRINT "invalid"; CONTINUE
      remaining <- units ; energy <- 0
      FOR each slab s
```

```
            in_slab <- MIN(remaining, SLAB_SIZE[s])
            energy <- energy + in_slab * SLAB_RATE[s]
            remaining <- remaining - in_slab
        energy <- energy + remaining * TOP_RATE
        IF units > 300 THEN surcharge <- 8% of energy ELSE surcharge <- 0
        IF units <= 100 THEN rebate <- 5% of energy ELSE rebate <- 0
        bill <- energy + FIXED + surcharge - rebate
        IF previous_bill > 0 THEN
            change <- (bill - previous_bill) / previous_bill * 100
            IF change > 25 THEN flag UNUSUAL RISE
        previous_bill <- bill ; update total and highest
        PRINT row
    END FOR
    PRINT total, average, highest, number of alerts
STOP
```

7. Python Source Code

```
# Case Study 2: Progressive Electricity Billing
# ---- INPUT DATA ----
units_used = [90, 150, 210, 320, 260, 410]
# ---- END DATA ----

SLAB_SIZE = [100, 100, 100]        # first three slabs (units)
SLAB_RATE = [3.50, 5.00, 7.00]     # rupees per unit in those slabs
TOP_RATE = 9.00                    # rupees per unit above 300 units
FIXED_CHARGE = 120.00
SURCHARGE_LIMIT = 300
SURCHARGE_PERCENT = 8
REBATE_LIMIT = 100
REBATE_PERCENT = 5
ALERT_PERCENT = 25

print("{:<8}{:>5}{:>9}{:>8}{:>8}{:>8}{:>9}{:>8}  {}".format(
      "Month", "Units", "Energy", "Fixed", "Surch.", "Rebate",
      "Bill", "Change", "Alert"))
print("-" * 74)

previous_bill = -1
alerts = 0
total_bill = 0.0
valid_months = 0
highest_bill = -1.0
highest_label = ""

for i in range(len(units_used)):
    units = units_used[i]
    label = "Month " + str(i + 1)
    if units < 0:
        print(label, ": invalid reading (", units, ") - skipped")
        continue

    remaining = units
    energy = 0.0
    for s in range(len(SLAB_SIZE)):
        in_slab = remaining
        if in_slab > SLAB_SIZE[s]:
            in_slab = SLAB_SIZE[s]
        energy = energy + in_slab * SLAB_RATE[s]
        remaining = remaining - in_slab
    energy = energy + remaining * TOP_RATE

    surcharge = 0.0
    if units > SURCHARGE_LIMIT:
        surcharge = energy * SURCHARGE_PERCENT / 100
    rebate = 0.0
    if units <= REBATE_LIMIT:
        rebate = energy * REBATE_PERCENT / 100
    bill = energy + FIXED_CHARGE + surcharge - rebate

    change_text = "-"
    alert = ""
    if previous_bill > 0:
        change = (bill - previous_bill) / previous_bill * 100
        change_text = "{:+.1f}%".format(change)
        if change > ALERT_PERCENT:
            alert = "UNUSUAL RISE"
            alerts = alerts + 1

    print("{:<8}{:>5}{:>9.2f}{:>8.2f}{:>8.2f}{:>8.2f}{:>9.2f}{:>8}  {}".format(
          label, units, energy, FIXED_CHARGE, surcharge, rebate,
          bill, change_text, alert))

    previous_bill = bill
    total_bill = total_bill + bill
    valid_months = valid_months + 1
    if bill > highest_bill:
        highest_bill = bill
        highest_label = label

print("-" * 74)
if valid_months == 0:
    print("No valid readings to bill.")
else:
    print("Total of all bills  : {:.2f}".format(total_bill))
    print("Average monthly bill: {:.2f}".format(total_bill / valid_months))
    print("Highest bill        : {} ({:.2f})".format(highest_label, highest_bill))
```

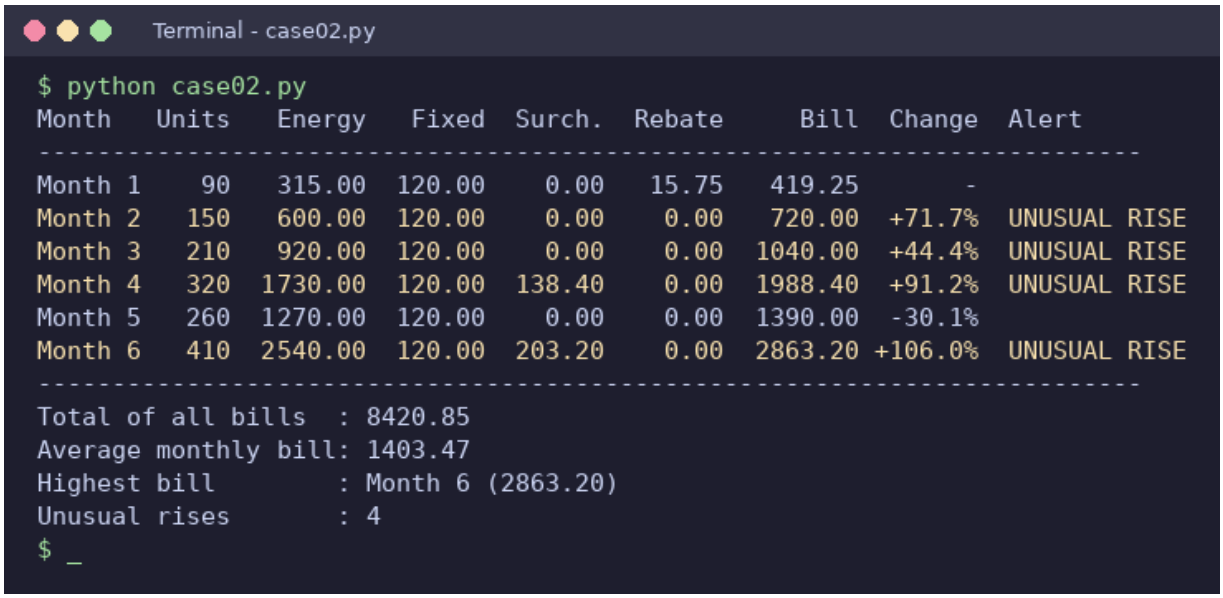
```
80      print("Unusual rises       :", alerts)
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Default six-month data	[90,150,210,320,260, 410]	Month 6 highest; 4 flagged rises	`Highest bill : Month 6 (2863.20) | Unusual rises : 4`	**PASS**
2	Rebate boundary (100 units)	[100]	350 + 120 - 17.50 = 452.50	`Month 1 100 350.00 120.00 0.00 17.50 452.50 -`	**PASS**
3	Top of third slab (300 units)	[300]	350+500+700+120 = 1670.00, no surcharge	`Month 1 300 1550.00 120.00 0.00 0.00 1670.00 -`	**PASS**
4	Just above surcharge limit (301 units)	[301]	1559 + 120 + 124.72 = 1803.72	`Month 1 301 1559.00 120.00 124.72 0.00 1803.72 -`	**PASS**
5	Zero consumption	[0]	Only the fixed charge is billed	`Month 1 0 0.00 120.00 0.00 0.00 120.00 -`	**PASS**
6	Invalid (negative) reading	[-5, 120]	Month 1 skipped; month 2 billed 570.00	`Month 1 : invalid reading ( -5 ) - skipped | Month 2 120 450.00 120.00 0.00 0.00 570.00 -`	**PASS**
7	No readings at all	[]	Graceful message	`No valid readings to bill.`	**PASS**

9. Output Screenshot

Terminal output of `python case02.py` using the default data:



10. Conclusion

The program reproduces a real progressive tariff using parallel lists for slab sizes and rates, adds the surcharge and rebate rules and detects abnormal month-to-month growth. Boundary tests at 100, 300 and 301 units confirm that slab edges and conditional rules are handled correctly.